AgentRelay — CLI & Systems Command Reference

Complete developer reference covering global installation, local initialization, opt-in capture scoping, background watcher daemons, offline search, MCP agent interop, team synchronization, sudo / zsh permission workarounds, and full data lifecycle management.

Target OS: macOS (zsh) / Linux	**Version:** 1.0.0 (Polaris PRD v0.1)	**Authors:** Utkarsh Sahoo & Daksh Vaswani
Store: SQLite + FTS5 (~/.agentrelay)	**Local Model:** Ollama (qwen2.5-coder:7b)	**Cloud:** LCP Express REST Server (/api/v1)

1. Installation & Global Setup (Including Sudo & Zsh Workarounds)

AgentRelay is a monorepo consisting of a Node.js CLI, an Express LCP backend, and a React dashboard. On macOS, installing global packages can occasionally encounter EACCES: permission denied if system directories are protected. Multiple install methods are supported below.

Method / Scenario	Command & Functional Description
`Standard Dependencies`	npm install --prefix cli npm install --prefix backend npm install --prefix frontend Installs all third-party dependencies across the workspace.
`Standard Global Link` *(Run inside project)*	cd ~/Desktop/DESKTOP/AgentRelay/cli && npm link && cd .. Symlinks the agentrelay binary into your system PATH.
`Global Install via Path`	npm install -g ~/Desktop/DESKTOP/AgentRelay/cli Directly installs the CLI globally from any working directory.
`EACCES Fix A: Sudo Global` *(Administrator)*	sudo npm install -g ~/Desktop/DESKTOP/AgentRelay/cli Overcomes macOS permission lock on /usr/local/lib/node_modules. Prompts for Mac password once.
`EACCES Fix B: Zsh Alias` *(Zero Sudo — Recommended)*	echo 'alias agentrelay="node ~/Desktop/DESKTOP/AgentRelay/cli/bin/agentrelay.js"' >> ~/.zshrc && source ~/.zshrc Permanent shell alias. Requires zero administrator privileges and works instantly in every terminal.
`EACCES Fix C: Chown Fix` *(Fix npm permissions)*	sudo chown -R $(whoami) /usr/local/lib/node_modules /usr/local/bin Permanently restores write ownership to your user account so npm never needs sudo again.
`Installation Verification`	agentrelay --version (Outputs: 1.0.0) agentrelay --help (Displays master command catalog)

2. Initialization & Opt-In Scope Management

By architectural design (PRD §8.1 FR-001, FR-002), AgentRelay enforces **strict opt-in scoping**. The default scope is completely empty; files and agent transcripts outside explicitly scoped folders are never opened.

Command	Functional Specification & Acceptance Criteria
`agentrelay init`	Creates ~/.agentrelay/ directory, initializes db.sqlite (WAL mode + FTS5 index), generates default config.json, verifies local model runtime (Ollama), and if authenticated, pulls team context records.

`agentrelay status`	Displays active daemon state, list of registered capture folders, total indexed sessions, scrubbed events count, distilled records count, distillation job queue depth, and model endpoint status.
`agentrelay scope add <path>`	Registers a folder path for transcript observation. The watcher begins tailing sessions inside .claude/, .cursor/, etc.
`agentrelay scope list`	Lists all directories currently registered for session capture.
`agentrelay scope remove <path>`	Removes a folder from observation. Ceases all transcript capture for that directory immediately.
`agentrelay pause`	Temporarily halts session watcher capture across all scopes (zero capture mode).
`agentrelay resume`	Resumes session watcher capture across all registered scopes.

3. Daemon, Background Execution & Distillation Pipeline

The background worker tails transcript files using inode-tracking and byte-offsets (≤ 10s latency), scrubs planted secrets deterministically, and runs the local distillation pipeline.

Command / Mode	Functionality & Shell Workflow
`agentrelay daemon`	Runs the continuous watcher, normalizer, secret scrubber, and distillation queue processor in the foreground.
`Zsh Background Daemon` *(Run in background)*	nohup agentrelay daemon > ~/.agentrelay/daemon.log 2>&1 & Runs daemon persistently in the background. Logs are written to ~/.agentrelay/daemon.log.
`Check Daemon Status`	ps aux | grep "agentrelay daemon" Checks if the background process is running.
`Stop Background Daemon`	kill $(pgrep -f "agentrelay daemon") Gracefully terminates the running background worker.
`agentrelay retry [sessionId]`	Re-queues distillation jobs that encountered model timeouts or transient errors.

4. Review, Feedback, Search & Record Sharing

Command	Functionality & Options
`agentrelay review`	Interactive CLI review of candidate distilled records. Supports **accept**, **edit** (creates new version), and **reject**.
`agentrelay review --accept`	Batch-accepts all unreviewed candidate records into the verified store.
`agentrelay review --reject`	Batch-rejects all candidate records, removing them from search and promotion.
`agentrelay search [query]`	Performs high-speed offline search over sessions, decisions, tags, and files touched using SQLite FTS5.
`agentrelay search [query] --json`	Outputs matching context records in structured JSON schema.
`agentrelay show <recordId>`	Displays full record summary, decisions citing evidence IDs, and files touched. Supports full UUID or prefix.
`agentrelay show <recordId> --json`	Outputs full record JSON including provenance and sensitivity metadata.
`agentrelay share <recordId>`	Marks a record as shareable with your team. (Records are strictly **private by default** per PRD FR-015).
`agentrelay unshare <recordId>`	Reverts a record back to private status, preventing promotion.

5. Global Context Sync, Markdown Packs & MCP Server (Agent Interop)

AgentRelay bridges human-reviewed team decisions directly to AI coding agents (Claude Code, Cursor, Windsurf, Aider) via file-based sync and standard Model Context Protocol (MCP) over stdio.

Command	Functionality & Agent Interoperability
`agentrelay sync`	**1-Command Master Synchronization:** 1. Pulls all team context records from the LCP server via sequence cursor. 2. Extracts and catalogs all global code files referenced across sessions. 3. Exports individual Markdown context packs to ~/.agentrelay/context/record_*.md. 4. Compiles ~/.agentrelay/context/CONTEXT.md and project-level .agentrelay/CONTEXT.md.
`agentrelay export <recordId>`	Exports a single record as a self-contained Markdown context pack (context_pack_*.md).
`agentrelay export <recordId> --pack`	Includes the full raw scrubbed session chat and tool event slice in the Markdown pack.
`agentrelay mcp`	**Model Context Protocol (MCP) Stdio JSON-RPC Server (FR-027):** Exposes read-only tools to IDE coding assistants: search_context (query past decisions and touched code) and show_record (retrieve full evidence context pack).

Claude Desktop / Cursor Configuration (claude_desktop_config.json):

```
{
"mcpServers": {
"agentrelay": {
"command": "agentrelay",
"args": ["mcp"]
}
}
}
```

6. Team Collaboration, Handoffs & LCP Cloud (PRD §8.2)

Lightweight Context Protocol (LCP) REST server featuring tenant isolation, cryptographic SHA-256 audit chaining, and teammate handoffs.

Command	Functionality & API Endpoints
`agentrelay login`	Authenticates with the LCP team server via GitHub OAuth device flow. Stores revocable API token in ~/.agentrelay/auth.json (0o600).
`agentrelay login --token <tok>`	Authenticates directly using an existing LCP / GitHub API access token.
`agentrelay logout`	Clears local credentials from auth.json.
`agentrelay team create <name>`	Creates a new isolated team workspace (sets creator as admin).
`agentrelay team list`	Lists all teams the user belongs to along with member rosters.
`agentrelay team invite`	Generates an expiring, single-use cryptographic team invite token.
`agentrelay team join <token>`	Joins an existing team workspace using an invite token.
`agentrelay promote <recordId>`	Promotes a shareable record to the team server. The server runs a **mandatory re-scrubbing pass** on ingest and appends an event to the SHA-256 audit chain.
`agentrelay unpromote <recordId>`	Withdraws a record from the team. Creates an immutable tombstone that propagates on pull.
`agentrelay pull`	Fetches new team records and tombstones incrementally using a monotonic sequence cursor.
`agentrelay handoff <recId> <@user>`	Routes a context record directly to a teammate's inbox with notification provenance.
`agentrelay inbox [options]`	Lists received context handoffs. Flags: --state pending|read, --read <id>, --json.

7. Backend Server & Web Companion Dashboard

Target Workspace	Command & Operational Purpose
`Backend Dev Server`	npm run dev --prefix backend Runs the Express LCP REST server on **port 4000** with Nodemon live-reloading.
`Backend Production`	npm run start --prefix backend Runs the backend server in production mode.
`Backend Seed`	npm run seed --prefix backend Populates lcp.db with mock teams, users, and context records.
`Frontend Dev Server`	npm run dev --prefix frontend Launches Vite HMR companion dashboard on **port 5173**.
`Frontend Production Build`	npm run build --prefix frontend Compiles optimized static production assets into frontend/dist/.
`Frontend Preview`	npm run preview --prefix frontend Locally previews the compiled production build.

8. Automated Testing, Isolation & Security Benchmarks

Test Suite / Benchmark	Command & Coverage
`CLI Unit Test Suite`	npm test --prefix cli 9 tests: scrubber, normalizer, adapter, schema validation, FR-011 sensitivity classifier, context exporter, and MCP JSON-RPC protocol.
`Backend Security Suite`	npm test --prefix backend 41 tests: multi-tenant isolation, GitHub OAuth, rate limiting, record lifecycle, tombstones, handoff inbox, and SHA-256 tamper-evident audit chain.
`CLI E2E Smoke Pipeline`	node cli/test/e2e-smoke.js 15-step headless end-to-end integration test creating a temporary project, capturing, scrubbing, distilling, reviewing, sharing, exporting, syncing, and querying MCP.
`Scrubber Metric M-1 Benchmark`	node eval/benchmark_scrubber.js Evaluates secret redaction recall against 24 planted secrets (AWS keys, GitHub PATs, AI keys, connection strings, private keys). **100% recall target achieved.**
`All-in-One Verification` ***(CI / Defense Command)***	npm test --prefix cli && npm test --prefix backend && node cli/test/e2e-smoke.js && node eval/benchmark_scrubber.js && npm run build --prefix frontend Executes the entire automated test and build pipeline in a single command.

9. Purge, Reset & Deletion / Uninstallation

Target Component	Command & Deletion Scope
`agentrelay purge --events`	Purges only the raw scrubbed events table from local SQLite (reclaims disk space while preserving distilled records).
`agentrelay purge --all`	Purges all local sessions, events, jobs, records, and byte offsets from the SQLite store.
`Reset Server Database`	rm -f backend/lcp.db backend/lcp.db-wal backend/lcp.db-shm Wipes the backend SQLite database to return the team server to a clean state.
`Delete Local Client Store`	rm -rf ~/.agentrelay Deletes all client configuration, tokens, local SQLite databases, and exported context packs.
`Delete Repository Context`	rm -rf .agentrelay/ Removes project-level CONTEXT.md files from current working repository.
`Global Uninstall (npm link)`	cd ~/Desktop/DESKTOP/AgentRelay/cli && npm unlink -g agentrelay && cd .. Removes global symlink from /usr/local/bin.
`Global Uninstall (npm -g)`	npm uninstall -g agentrelay-cli Uninstalls globally installed package.
`Remove Zsh Shell Alias`	sed -i '' '/agentrelay/d' ~/.zshrc && source ~/.zshrc Removes the alias from your zsh configuration file.